

NATIONAL ACADEMY OF PROFESSIONAL STUDIES

ITS203 • ASSESSMENT C • MILESTONE 1

StockMate

Project Idea Proposal

A simple stock tracker for a small motel

Student Name	Aashir Bhusal
Student ID	s2400901
Unit Code	ITS203
Unit Name	Object-Oriented Design and Programming
Assessment	Assessment C — Milestone 1: Project Idea Proposal (5%)
Tools	C# / Windows Forms, SQLite, Visual Studio
Due	Week 7 — 28 August 2026, 11:59 pm AEST

Submitted in accordance with NAPS Student Academic Integrity and Misconduct Policy A009.

1. Problem description

Small motels go through a lot of stock every day: towels, sheets, soap, toilet paper, cleaning products and breakfast items. Most small motels do not track this properly. The stock count is written on a piece of paper in the storeroom, or it is not written down at all.

This causes two problems. The first is running out. If housekeeping finds there are no clean towels at 9am, someone has to drive to the shops and buy them at full price. That wastes both money and time. The second problem is ordering too much. When nobody knows how much is left, staff order extra to be safe. That stock then sits in the storeroom, and items like milk and hand sanitiser go out of date before anyone uses them.

The people who would use StockMate are the motel manager and the housekeeping supervisor. They are not computer experts. They need to see three things quickly: what is in stock, what is about to run out, and who took it. Programs like this already exist, but they are built for large hotel chains and cost far too much for a small motel.

2. Proposed idea

StockMate is a Windows desktop program written in C# using Windows Forms. It keeps a count of every stock item at one motel.

When the program opens, it shows a list of all items and how many are left. At the top it shows two warnings: items that are running low, and items that are close to their use-by date.

Staff record two kinds of movement. When a delivery arrives they record a **receipt** and the count goes up. When they take stock out of the storeroom they record an **issue** and the count goes down. Each movement saves the date and the staff member's name, so you can see where the stock went.

The program splits stock into two types, because they behave differently. **Consumable** items such as milk and soap have a use-by date. **Durable** items such as kettles and towels do not go off, but they wear out and need replacing after a while. Both are stock items and share most of their code, but each type decides in its own way whether it needs attention. This is why the program uses an abstract `StockItem` class with two child classes, `ConsumableItem` and `DurableItem`.

3. Inspiration

I picked this project because I have seen the problem myself. I help out at the Marco Polo Motor Inn in Sydney, where the storeroom is managed on paper. I have seen staff drive to the shops in the middle of a shift because the towels ran out, and I have seen breakfast stock thrown away because nobody knew it was there until it was already out of date.

I also picked it because it suits object-oriented programming. Having two kinds of stock that behave differently gives me a real reason to use inheritance and polymorphism, instead of adding classes just to tick a box.

4. Functional requirements

FR-01 to FR-06 are the main features I will build. FR-07 and FR-08 are extras I will add only if I have time.

ID	What the program will do	Priority
FR-01	The user can add a new stock item with a name, category, unit, starting quantity, reorder level and unit cost, and choose whether it is consumable or durable.	Main
FR-02	The user can edit an item, or turn it off so it no longer shows in the list without losing its history.	Main
FR-03	The user can record a delivery (receipt), which adds to the quantity and saves the date and staff name.	Main
FR-04	The user can record stock going out (issue), which subtracts from the quantity. The program refuses to take out more than there is, and explains why.	Main

ID	What the program will do	Priority
FR-05	The program shows all items in a table with name, category, quantity and reorder level, and the user can filter by category.	Main
FR-06	The program shows a warning list when the program opens: items at or below their reorder level, and consumables expiring within 30 days.	Main
FR-07	The user can see the full history for one item, showing each receipt and issue with its date, quantity and staff name.	Extra
FR-08	The user can export the current stock list, with a total value, to a CSV file.	Extra

Table 1 — Functional requirements

5. Non-functional requirements

Performance

The program should feel fast for one motel, which is up to about 2,000 items. The list should appear within one second of opening the program, and filtering or searching should not pause. Saving a receipt or an issue should take less than one second.

Usability

The users are motel staff, not computer experts, and there will be no training. A new user should be able to add an item and record a delivery without instructions. Everything will be in one window with tabs. Number boxes will not accept letters, and the program will ask "are you sure?" before anything that cannot be undone.

Reliability

The program must not lose stock data and must not crash on bad input. Each save runs as one database transaction, so a movement is either saved fully or not at all. Bad input or a missing database file will show a simple message box instead of an error screen.

6. Data storage plan

StockMate will use a **SQLite database** in one file called `stockmate.db`, saved next to the program.

I chose a database instead of a text file for three reasons. First, the data is linked: every movement belongs to an item, and a database keeps that link correct for me. Second, the warnings and the total value are easy to work out with a short SQL query, but would need a lot of code if I looped through a text file. Third, recording an issue changes two tables at once, and a database transaction makes sure both changes save together or neither does. I chose SQLite instead of MySQL or SQL Server because it does not need a server to be installed, so the program can just be copied onto a motel computer and run.

Table	Fields	What it stores
StockItems	ItemId (PK), Name, Category, ItemType, Unit, QuantityOnHand, ReorderLevel, UnitCost, IsActive, ExpiryDate, ReplacementMonths, LastReplacedDate	One row per item. <code>ItemType</code> is either 'Consumable' or 'Durable', which tells the program which class to build when it loads the row. <code>ExpiryDate</code> is only used by consumables, and the last two fields only by durables.
Transactions	TransactionId (PK), ItemId (FK), TransactionType, Quantity, TransactionDate, StaffName	Every movement in or out. <code>TransactionType</code> is either 'Receipt' or 'Issue'. These rows are never changed or deleted, so the history stays correct.

Table 2 — Database tables

7. Scope

I will build FR-01 to FR-06, which is a working program for one user on one computer. I am leaving out multiple users, ordering from suppliers, barcode scanning, accounting links, and any website or phone app. If I run short of time I will drop FR-08 first, then FR-07. I will commit my work to GitHub as I go rather than uploading it all at the end, and I will include the repository link with Milestone 2.